

Table 3: Orchestration and scaling with Docker tools

Aspect	Description	Additional details / Examples
Docker Swarm	Native Docker clustering and orchestration tool integrated with Docker CLI.	Easy to enable with <i>docker swarm init</i> ; supports simple <i>YAML Compose</i> files. Ideal for small to medium setups.
Kubernetes	Powerful, widely adopted orchestration platform with a rich feature set for large-scale container management.	Uses declarative <i>YAML</i> manifests (deployment, service, ingress) and supports complex use cases.
Automated deployment	Containers are deployed and updated automatically based on configuration files describing desired state.	Kubernetes uses <i>kubectl apply -f deployment.yaml</i> for declarative deployment.
Scaling	Supports horizontal scaling by increasing container replicas; vertical scaling adjusts container resources.	Example: <i>kubectl scale deployment my-app --replicas=5</i> to scale out pods.
Load balancing and discovery	Incoming requests are distributed across multiple container instances; service discovery helps locate services dynamically.	Kubernetes provides internal DNS for services; Swarm uses built-in load balancing via routing mesh.
Self-healing	Automatically monitors container health and restarts or replaces containers that fail or become unresponsive.	Kubernetes probes (livenessProbe, readinessProbe) detect failures and trigger restarts.
Rolling updates and rollbacks	Allows zero-downtime deployment of new versions; if issues arise, rollbacks restore previous versions.	Kubernetes commands: <i>kubectl rollout status deployment/my-app</i> and <i>kubectl rollout undo deployment/my-app</i> .
Resource management	Define CPU and memory requests and limits to optimise usage and prevent resource contention.	Example <i>YAML</i> snippet: <i>resources: requests: cpu: "100m" memory: "128Mi"</i> limits resource usage per container.

```
#####
flask==2.3.2
#####
order-service/Dockerfile
#####
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
#####
docker-compose.yml (root folder)
#####
version: '3'
services:
user-service:
build: ./user-service
ports:
- "5000:5000"
order-service:
build: ./order-service
ports:
```

```
- "5001:5000"
#####
To build and run both services:
#####
docker-compose up --build
```

Service communication and discovery

Several independent services must communicate with one another effectively and consistently in a microservices architecture. Creating scalable and maintainable systems requires the design of efficient communication and discovery mechanisms.

Communication of services: The most popular methods for implementing synchronous communication are gRPC and RESTful APIs. Services call one another directly and watch for answers. Implementation is straightforward, but it may result in tight coupling and possible latency problems.

Asynchronous communication: Sends messages without waiting for instant answers by using message brokers such as Kafka, RabbitMQ, or AWS SQS. Enhances scalability and resilience through service decoupling and gracefully managing traffic spikes.